



Polars

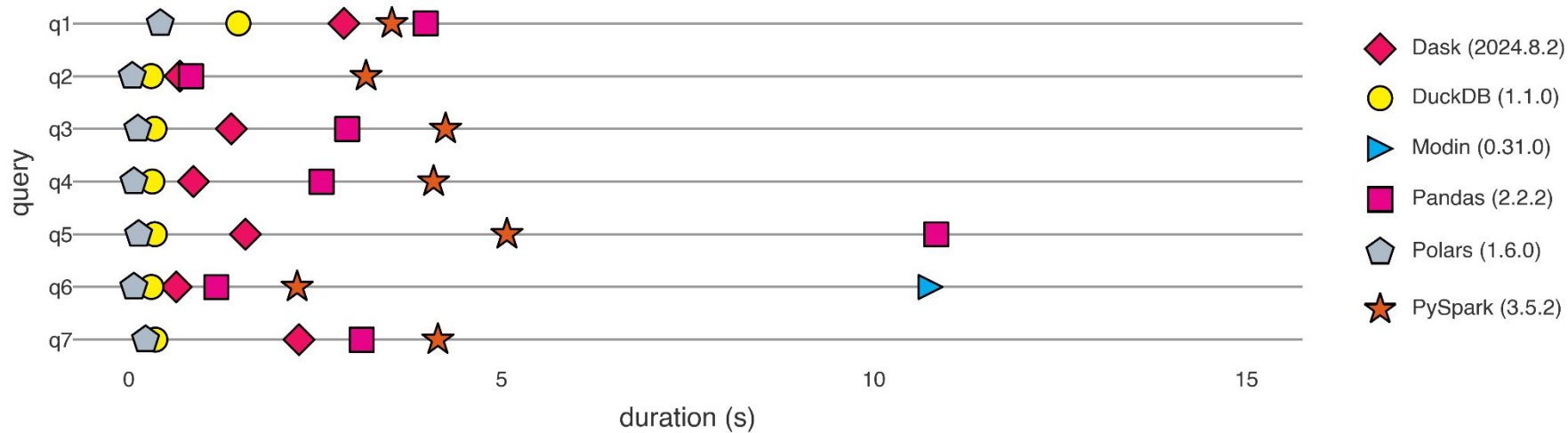
`posit::conf(2026)`

Polars

- DataFrame library
- By Ritchie Vink et al.
- Rust, Python, R, JavaScript
- Blazingly fast
- Expressive API

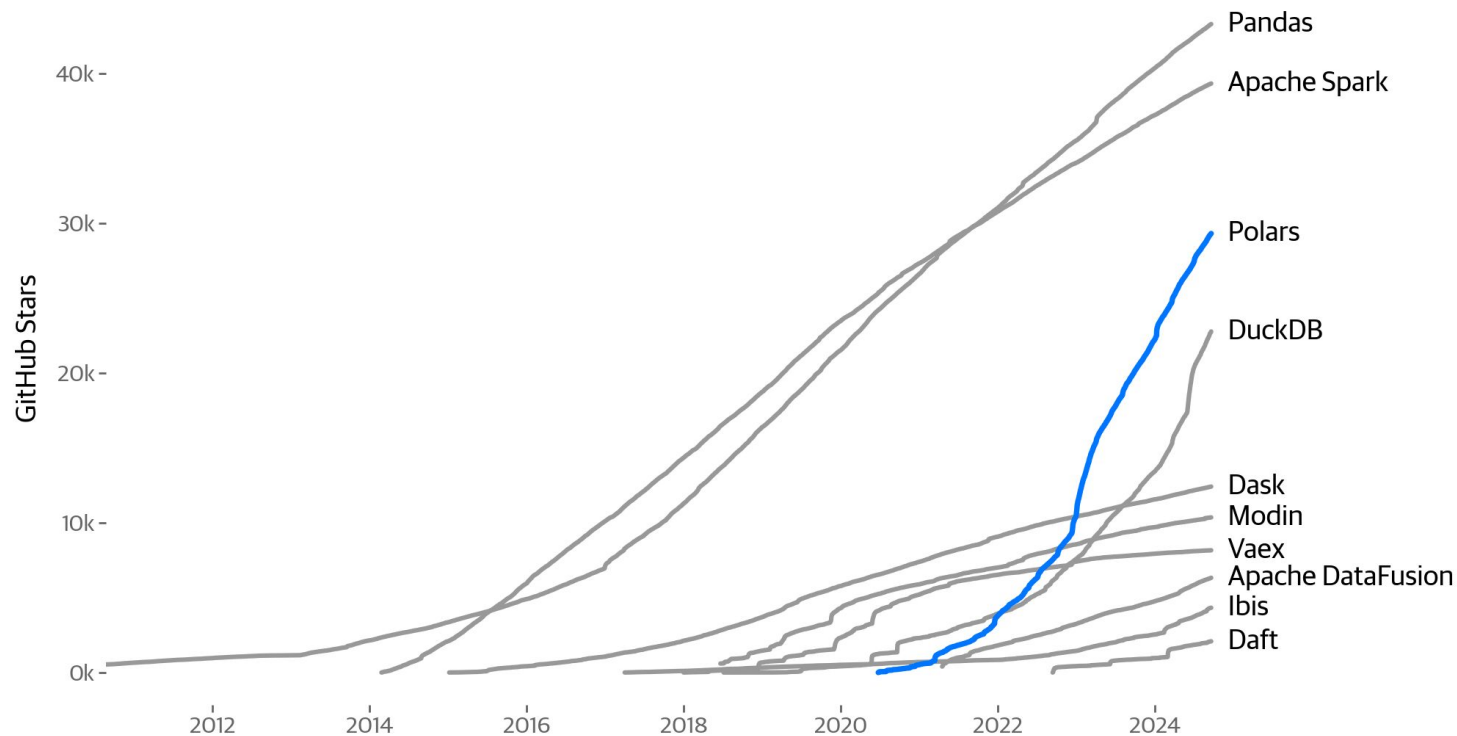


Polars is fast



Note: Modin took 21.6s on q1, took 23.9s on q2, took 28.4s on q3, took 51.0s on q4, took 171.2s on q5, and took 35.9s on q7. More information: <https://www.pola.rs/benchmarks.html>

Polars is popular



Source: GitHub, Retrieval Date: 2024-09-18

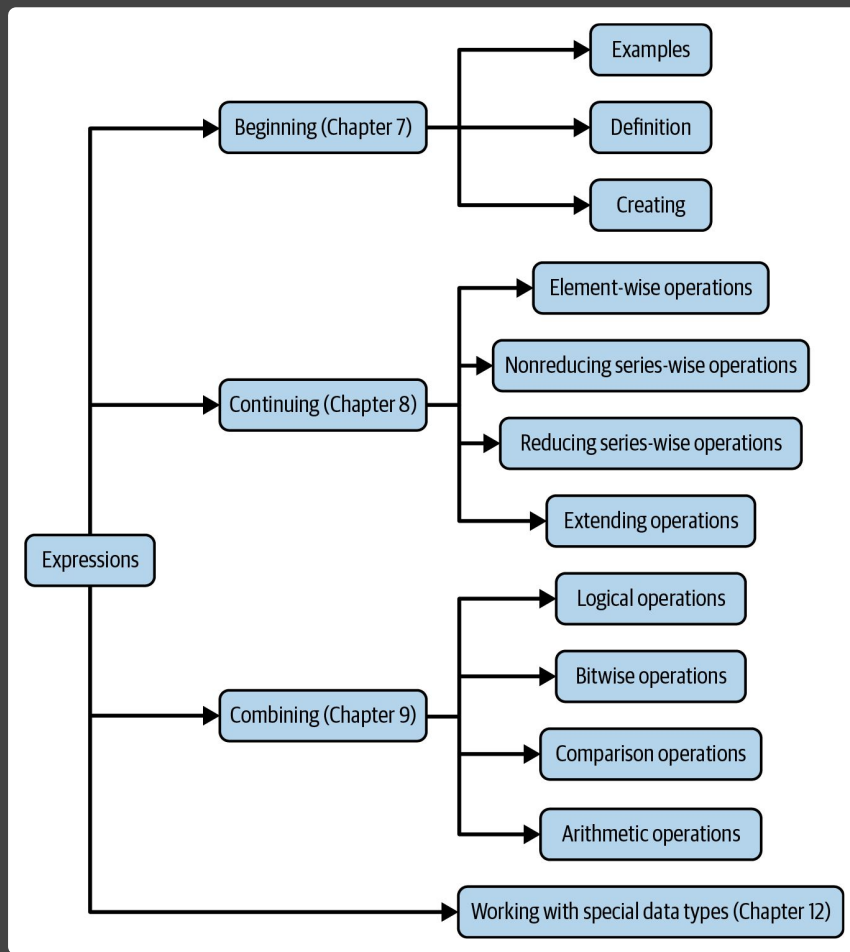
Polars. For the speed, come you must. For the API, stay you will.



Expressions are 50% of the API

ETL

```
>>> import polars as pl
>>> len(get_funcs(pl))
134
>>> len(get_funcs(pl.DataFrame))
250
>>> len(get_funcs(pl.Expr))
404
```



Let's look at some examples

```
>>> fruit = pl.read_csv("data/fruit.csv")
>>> fruit
shape: (10, 5)
```

name --- str	weight --- i64	color --- str	is_round --- bool	origin --- str
Avocado	200	green	false	South America
Banana	120	yellow	false	Asia
Blueberry	1	blue	false	North America
Cantaloupe	2500	orange	true	Africa
Cranberry	2	red	false	North America
Elderberry	1	black	false	Europe
Orange	130	orange	true	Asia
Papaya	1000	orange	false	South America
Peach	150	orange	true	Asia
Watermelon	5000	green	true	Africa

```
>>> fruit.select(
...     "name",
...     pl.col("^.*or.*$"),
...     pl.col("weight") / 1000,
...     "is_round",
... )
shape: (10, 5)
```

name --- str	color --- str	origin --- str	weight --- f64	is_round --- bool
Avocado	green	South America	0.2	false
Banana	yellow	Asia	0.12	false
Blueberry	blue	North America	0.001	false
Cantaloupe	orange	Africa	2.5	true
Cranberry	red	North America	0.002	false
Elderberry	black	Europe	0.001	false
Orange	orange	Asia	0.13	true
Papaya	orange	South America	1.0	false
Peach	orange	Asia	0.15	true
Watermelon	green	Africa	5.0	true

```
>>> fruit.with_columns(
...     pl.lit(True).alias("is_fruit"),
...     is_berry=pl.col("name").str.ends_with("berry"),
... )
shape: (10, 7)
```

name --- str	weight --- i64	color --- str	is_round --- bool	origin --- str	is_fruit --- bool	is_berry --- bool
Avocado	200	green	false	South Amer...	true	false
Banana	120	yellow	false	Asia	true	false
Blueberry	1	blue	false	North Amer...	true	true
Cantaloupe	2500	orange	true	Africa	true	false
Cranberry	2	red	false	North Amer...	true	true
Elderberry	1	black	false	Europe	true	true
Orange	130	orange	true	Asia	true	false
Papaya	1000	orange	false	South Amer...	true	false
Peach	150	orange	true	Asia	true	false
Watermelon	5000	green	true	Africa	true	false

```
>>> fruit.filter(  
...     (pl.col("weight") > 1000)  
...     & pl.col("is_round")  
... )  
shape: (2, 5)
```

name --- str	weight --- i64	color --- str	is_round --- bool	origin --- str
Cantaloupe	2500	orange	true	Africa
Watermelon	5000	green	true	Africa

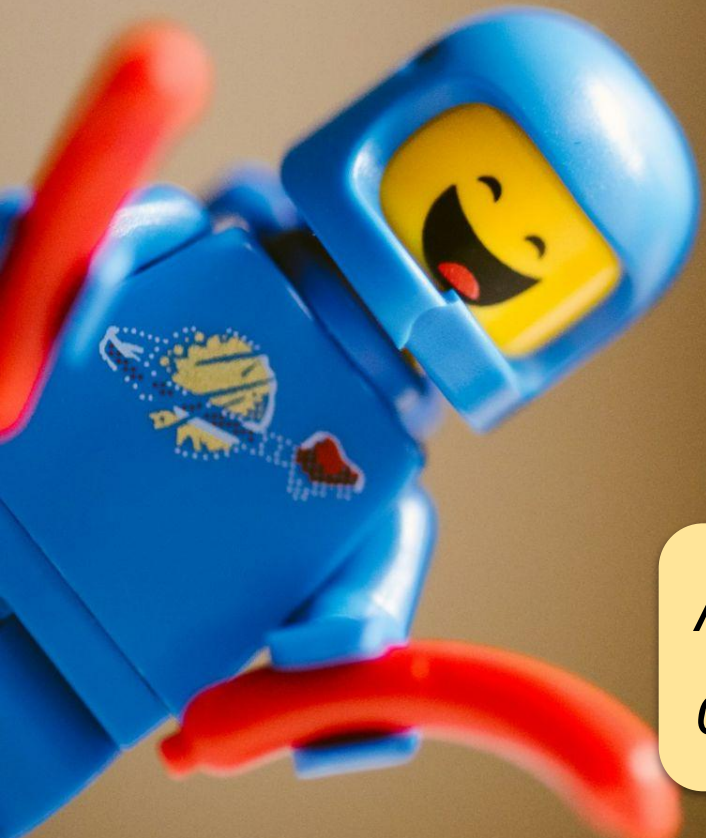
```
>>> fruit.groupby(pl.col("origin").str.split(" ").list.last()).agg(
...     pl.len(),
...     average_weight=pl.col("weight").mean()
... )
shape: (4, 3)
```

origin --- str	len --- u32	average_weight --- f64
America	4	300.75
Asia	3	133.333333
Africa	2	3750.0
Europe	1	1.0

```
>>> fruit.sort(
...     pl.col("name").str.len_bytes(),
...     descending=True,
... )
shape: (10, 5)
```

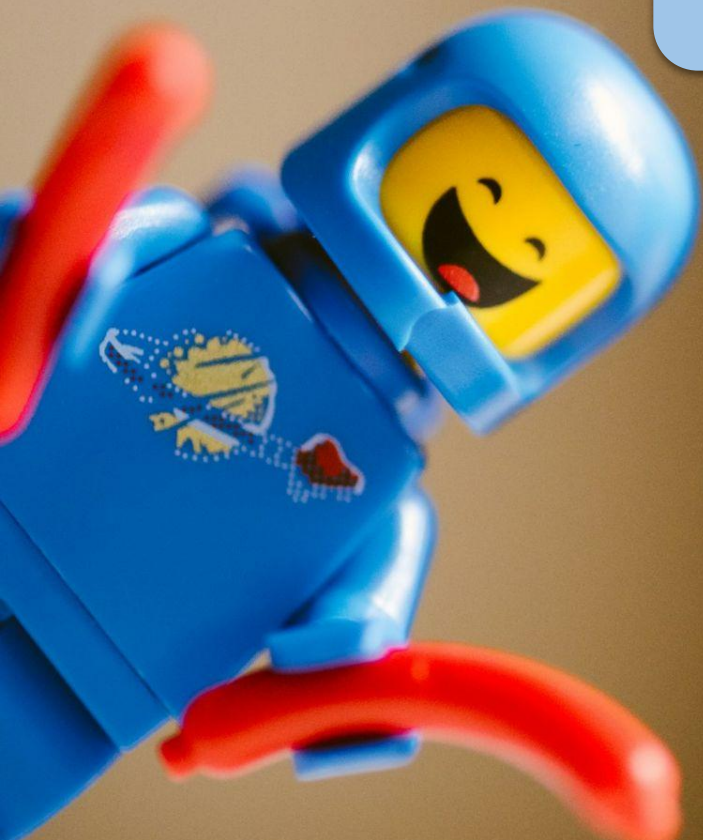
name --- str	weight --- i64	color --- str	is_round --- bool	origin --- str
Cantaloupe	2500	orange	true	Africa
Elderberry	1	black	false	Europe
Watermelon	5000	green	true	Africa
Blueberry	1	blue	false	North America
Cranberry	2	red	false	North America
Avocado	200	green	false	South America
Banana	120	yellow	false	Asia
Orange	130	orange	true	Asia
Papaya	1000	orange	false	South America
Peach	150	orange	true	Asia

What is an Expression?



*An expression is a lazy representation
of a data transformation.*

*An expression is a function
from a DataFrame to a Series.*



An expression is a tree of operations that describe how to construct one or more Series.



An expression is a tree of operations that describe how to construct one or more Series.



An expression is a tree of operations that describe how to construct one or more Series.



```
>>> (p1.lit(3).add(5) / p1.lit(1).add(5)).meta.tree_format()
```

0

1

2

3

0

binary: /

1

binary: +

binary: +

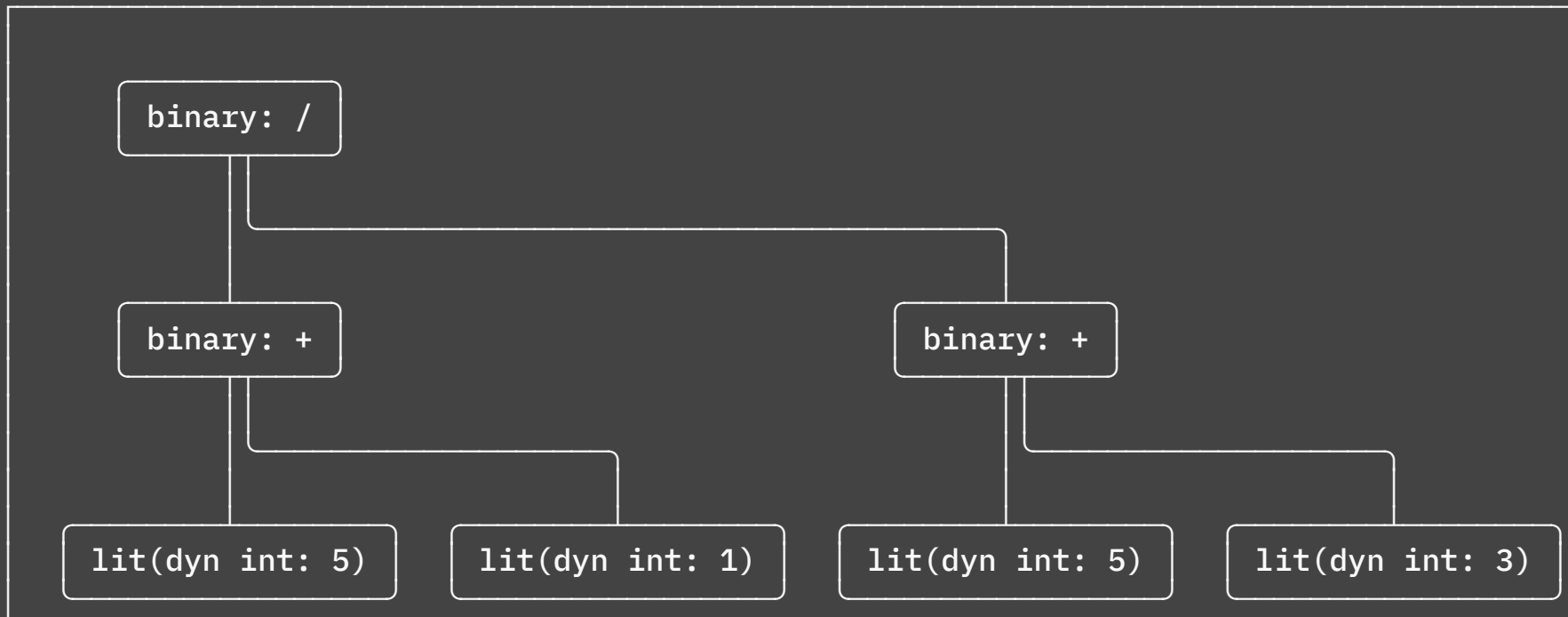
2

lit(dyn int: 5)

lit(dyn int: 1)

lit(dyn int: 5)

lit(dyn int: 3)



An expression is a tree of operations that describe how to construct one or more Series.



*An expression is a tree of operations that describe how to **construct** one or more Series.*



*An expression is a tree of operations that describe how to construct **one or more** Series.*



```
>>> (
...     pl.DataFrame({"a": [1, 2, 3], "b": [0.4, 0.5, 0.6]}).with_columns(
...         pl.all().mul(10).name.suffix("_times_10")
...     )
... )
shape: (3, 4)
```

a --- i64	b --- f64	a_times_10 --- i64	b_times_10 --- f64
1	0.4	10	4.0
2	0.5	20	5.0
3	0.6	30	6.0

Properties of Expressions

Properties of Expressions

- **Lazy**
- Function and data dependent
- Reusable
- Expressive
- Efficient
- Idiomatic

Properties of Expressions

- Lazy
- **Function and data dependent**
- Reusable
- Expressive
- Efficient
- Idiomatic

```
>>> is_orange = (pl.col("color") == "orange").alias("is_orange")
>>> fruit.with_columns(is_orange)
shape: (10, 6)
```

name --- str	weight --- i64	color --- str	is_round --- bool	origin --- str	is_orange --- bool
Avocado	200	green	false	South America	false
Banana	120	yellow	false	Asia	false
Blueberry	1	blue	false	North America	false
Cantaloupe	2500	orange	true	Africa	true
Cranberry	2	red	false	North America	false
Elderberry	1	black	false	Europe	false
Orange	130	orange	true	Asia	true
Papaya	1000	orange	false	South America	true
Peach	150	orange	true	Asia	true
Watermelon	5000	green	true	Africa	false

```
>>> fruit.filter(is_orange)
shape: (4, 5)
```

name --- str	weight --- i64	color --- str	is_round --- bool	origin --- str
Cantaloupe	2500	orange	true	Africa
Orange	130	orange	true	Asia
Papaya	1000	orange	false	South America
Peach	150	orange	true	Asia

```
>>> fruit.groupby(is_orange).len()
shape: (2, 2)
```

is_orange --- bool	len --- u32
false	6
true	4

Properties of Expressions

- Lazy
- Method dependent
- **Reusable**
- Expressive
- Efficient
- Idiomatic

```
>>> flowers = pl.DataFrame(  
...     {  
...         "name": ["Tiger lily", "Blue flag", "African marigold"],  
...         "latin": ["Lilium columbianum", "Iris versicolor", "Tagetes erecta"],  
...         "color": ["orange", "purple", "orange"],  
...     }  
... )  
>>> flowers.filter(is_orange)  
shape: (2, 3)
```

name --- str	latin --- str	color --- str
Tiger lily African marigold	Lilium columbianum Tagetes erecta	orange orange

Properties of Expressions

- Lazy
- Function and data dependent
- Reusable
- **Expressive**
- Efficient
- Idiomatic

Properties of Expressions

- Lazy
- Function and data dependent
- Reusable
- Expressive
- **Efficient**
- Idiomatic

Properties of Expressions

- Lazy
- Function and data dependent
- Reusable
- Expressive
- Efficient
- **Idiomatic**

```
>>> fruit.filter((fruit["weight"] > 1000) & fruit["is_round"])  
shape: (2, 5)
```

name --- str	weight --- i64	color --- str	is_round --- bool	origin --- str
Cantaloupe	2500	orange	true	Africa
Watermelon	5000	green	true	Africa

```
>>> (
...     fruit.lazy()
...     .filter((pl.col("weight") > 1000) & pl.col("is_round"))
...     .with_columns(is_berry=pl.col("name").str.ends_with("berry"))
...     .collect()
... )
shape: (2, 6)
```

name --- str	weight --- i64	color --- str	is_round --- bool	origin --- str	is_berry --- bool
Cantaloupe	2500	orange	true	Africa	false
Watermelon	5000	green	true	Africa	false

```
>>> (
...     fruit.lazy()
...     .filter((fruit["weight"] > 1000) & fruit["is_round"])
...     .with_columns(is_berry=fruit["name"].str.ends_with("berry"))
...     .collect()
... )
```

ShapeError: unable to add a column of length 10 to a DataFrame of height 2

Expressions: Beyond Polars



ENH: Introduce `pandas.col` #62103

New issue

Merged mroeschke merged 33 commits into pandas-dev:main from MarcoGorelli:pandas-col last week

Conversation 32

Commits 33

Checks 42

Files changed 9

+423 -3



MarcoGorelli commented 3 weeks ago · edited

Member ...

xref @jbrockmendel's comment #56499 (comment)

I'd also discussed this with @phofl, @WillAyd, and @jorisvandenbossche (who originally showed us something like this in Basel at euroscipy 2023)

Demo:

```
import pandas as pd
from datetime import datetime

df = pd.DataFrame(
    {
        "a": [1, -2, 3],
        "b": [4, 5, 6],
        "c": [datetime(2020, 1, 1), datetime(2025, 4, 2), datetime(2026, 12, 3)],
        "d": ["fox", "beluga", "narwhal"],
    }
)

result = df.assign(
    # The usual Series methods are supported
    a_abs=pd.col("a").abs(),
    # And can be combined
    a_centered=pd.col("a") - pd.col("a").mean(),
    a_plus_b=pd.col("a") + pd.col("b"),
    # Namespace are supported too
    c_year=pd.col("c").dt.year,
    c_month_name=pd.col("c").dt.strftime("%B"),
    d_upper=pd.col("d").str.upper(),
).loc[pd.col("a_abs") > 1] # This works in `loc` too

print(result)
```

Reviewers

Dr-Irv ✓

mroeschke ✓

+1 more reviewer ^

dangotbanned

Assignees

No one assigned

Labels

Enhancement

Projects

None yet

Milestone

3.0

Development

Successfully merging this pull request may close these issues.

None yet

Conclusion

- Expressions are 50% of the API
- Expressions are elegant
- Expressions are important

Thank you



Resources

- [Python Polars: The Definitive Guide](#)
- [Data Science at the Command Line, 2e](#)
- [Table and Plotnine Contests - Posit](#)
- [Narwhals](#)
- [ENH: Introduce pandas.col #62103](#)

